

k29photo Portal — Αναφορά Άσκησης 3

Κ29: Σχεδιασμός και Χρήση Βάσεων Δεδομένων — Εαρινό Εξάμηνο 2026 Τμήμα Πληροφορικής & Τηλεπικοινωνιών, ΕΚΠΑ

ΟΝΟΜΑ : ADAM AHMED
Α.Μ. : 1115202400225

1. Εισαγωγή

Στην άσκηση αυτή σχεδίασα και υλοποίησα το k29photo, μια πλήρως λειτουργική διαδικτυακή πλατφόρμα κοινοποίησης φωτογραφιών εμπνευσμένη από το Flickr. Το σύστημα υλοποιήθηκε αποκλειστικά με PostgreSQL ως μηχανή βάσης δεδομένων και Flask (Python 3) ως web framework, σύμφωνα με τις απαιτήσεις της εκφώνησης. Όλες οι αλληλεπιδράσεις με τη βάση δεδομένων πραγματοποιούνται μέσω raw SQL εντολών χρησιμοποιώντας τη βιβλιοθήκη `psycopg2` — δεν χρησιμοποιήθηκε κανένα ORM.

Η πλατφόρμα υποστηρίζει εγγραφή και αυθεντικοποίηση χρηστών, διαχείριση άλμπουμ και φωτογραφιών, περιήγηση και αναζήτηση μέσω ετικετών, σχόλια από εγγεγραμμένους χρήστες και επισκέπτες, likes, και εξατομικευμένες συστάσεις φίλων και φωτογραφιών.

2. Περιγραφή Διαγράμματος E/R

2.1 Οντότητες και Χαρακτηριστικά

Η βάση δεδομένων αποτελείται από οκτώ πίνακες. Οι πέντε κύριες οντότητες είναι Users, Albums, Photos, Tags και Comments. Τρεις πίνακες σύνδεσης διαχειρίζονται τις σχέσεις πολλά-προς-πολλά: photo_tags, friends και likes.

Πίνακας	Κύρια Χαρακτηριστικά	Πρωτεύον Κλειδί
users	first_name, last_name, email (UNIQUE), dob, hometown, gender, password	user_id SERIAL
albums	name, owner_id (FK→users), creation_date	album_id SERIAL
photos	album_id (FK→albums), caption, data (BYTEA)	photo_id SERIAL

tags	tag_name (UNIQUE, πεζά, χωρίς κενά)	tag_id SERIAL
comments	photo_id (FK), user_id (FK nullable), guest_name, content, post_date	comment_id SERIAL
friends	user_id (FK→users), friend_id (FK→users)	σύνθετο PK
photo_tags	photo_id (FK→photos), tag_id (FK→tags)	σύνθετο PK
likes	user_id (FK→users), photo_id (FK→photos)	σύνθετο PK

2.2 Σχέσεις

- User owns Albums (1:N) — ένας χρήστης μπορεί να κατέχει πολλά άλμπουμ. Η διαγραφή χρήστη διαδίδεται αυτόματα σε άλμπουμ, φωτογραφίες, σχόλια και likes μέσω ON DELETE CASCADE.
- Album contains Photos (1:N) — κάθε φωτογραφία ανήκει ακριβώς σε ένα άλμπουμ. Δεν επιτρέπεται φωτογραφία χωρίς άλμπουμ.
- User follows User (M:N, κατευθυντική) — υλοποιείται μέσω του πίνακα friends. Το ότι ο A ακολουθεί τον B δεν σημαίνει ότι ο B ακολουθεί τον A. Η αμοιβαία ακολούθηση επιτρέπεται εάν και οι δύο χρήστες επιλέξουν να το κάνουν.
- Photo tagged with Tags (M:N) — μέσω του πίνακα photo_tags. Μια φωτογραφία μπορεί να έχει μηδέν ή περισσότερες ετικέτες, και μια ετικέτα μπορεί να αντιστοιχεί σε πολλές φωτογραφίες από διαφορετικά άλμπουμ.
- User likes Photo (M:N) — μέσω του πίνακα likes. Κάθε ζεύγος (user_id, photo_id) είναι μοναδικό, εξασφαλίζοντας ότι κάθε χρήστης μπορεί να κάνει like σε μια φωτογραφία μόνο μία φορά.
- User writes Comment on Photo — τα σχόλια αναφέρονται σε μια φωτογραφία και προαιρετικά σε εγγεγραμμένο χρήστη. Για επισκέπτες το user_id είναι NULL και αποθηκεύεται το guest_name.

3. Περιορισμοί και Triggers

3.1 CHECK Περιορισμοί

- users.email — UNIQUE: Η προσπάθεια εγγραφής με διπλότυπη διεύθυνση email παράγει μήνυμα σφάλματος που εμφανίζεται στον χρήστη.

- `users.gender` — CHECK (`gender IN ('M', 'F', 'O')`): Γίνονται δεκτοί μόνο έγκυροι κωδικοί φύλου σε επίπεδο βάσης δεδομένων.
- `tags.tag_name` — UNIQUE + CHECK: Επιβάλλεται η χρήση πεζών γραμμάτων (`tag_name = LOWER(tag_name)`) και η απουσία κενών (`tag_name NOT LIKE '% %'`), σύμφωνα με την εκφώνηση.
- `comments` — CHECK `commenter`: Ακριβώς ένα από τα δύο πεδία `user_id` ή `guest_name` πρέπει να είναι non-null. Αυτό εξασφαλίζει ότι κάθε σχόλιο έχει πάντα αναγνωρίσιμο συγγραφέα.
- `friends` — CHECK (`user_id ≠ friend_id`): Αποτρέπεται η αυτο-φιλία σε επίπεδο βάσης.

3.2 Triggers

- `trg_no_self_comment`: Εκτελείται BEFORE INSERT στον πίνακα `comments`. Εντοπίζει τον ιδιοκτήτη του άλμπουμ της φωτογραφίας και δημιουργεί εξαίρεση εάν εγγεγραμμένος χρήστης προσπαθήσει να σχολιάσει τη δική του φωτογραφία, όπως απαιτεί η εκφώνηση.
- `trg_no_self_like`: Εκτελείται BEFORE INSERT στον πίνακα `likes`. Αρχικά υλοποιήθηκε ως trigger, αλλά τελικά αποφάσισα να επιτρέψω το like στις δικές μας φωτογραφίες καθώς η εκφώνηση δεν αναφέρει ρητά αυτόν τον περιορισμό.

3.3 Κανόνες Cascade

Όλα τα ξένα κλειδιά χρησιμοποιούν ON DELETE CASCADE. Η διαγραφή χρήστη αφαιρεί αυτόματα τα άλμπουμ του, τα οποία αφαιρούν τις φωτογραφίες τους, οι οποίες αφαιρούν όλα τα σχόλια, likes και `photo_tags` που σχετίζονται. Αυτό εξασφαλίζει αναφορική ακεραιότητα χωρίς χειροκίνητη διαγραφή.

3.4 Indexes

Δημιούργησα indexes σε όλες τις στήλες ξένων κλειδιών που χρησιμοποιούνται σε JOIN λειτουργίες: `idx_albums_owner`, `idx_photos_album`, `idx_comments_photo`, `idx_comments_user`, `idx_photo_tags_tag`, `idx_photo_tags_photo`, `idx_friends_user`, `idx_likes_photo`. Αυτοί βελτιώνουν σημαντικά την απόδοση των πιο συχνών ερωτημάτων, ιδιαίτερα της αναζήτησης φωτογραφιών μέσω ετικετών και της εύρεσης σχολίων ανά φωτογραφία.

4. Παραδοχές Σχεδιασμού

Κατά τη διάρκεια της υλοποίησης έκανα τις παρακάτω παραδοχές για σημεία που δεν προσδιορίζονταν με ακρίβεια στην εκφώνηση:

- Κατευθυντική φιλία (μοντέλο follow). Η εκφώνηση αναφέρει «προσθήκη φίλου στη λίστα φίλων». Αυτό το ερμήνευσα ως κατευθυντική σχέση — ο χρήστης A μπορεί να ακολουθεί τον B χωρίς ο B να ακολουθεί τον A. Η αμοιβαία ακολούθηση είναι εφικτή αλλά δεν είναι αυτόματη.

- Σχόλια επισκεπτών. Η περιγραφή δεδομένων της εκφώνησης αναφέρει ότι ο ιδιοκτήτης σχολίου πρέπει να είναι «νόμιμος χρήστης του συστήματος», ενώ η περίπτωση χρήσης 4α επιτρέπει ρητά σχόλια από επισκέπτες. Ακολούθησα την περίπτωση χρήσης ως πιο ειδική: το `user_id` είναι nullable και υπάρχει πεδίο `guest_name`, με CHECK constraint που εξασφαλίζει ότι πάντα υπάρχει ένας από τους δύο.
 - Ετικέτες καθολικές. Οι ετικέτες είναι κοινές για όλους τους χρήστες και φωτογραφίες, όχι ανά χρήστη. Αυτό είναι απαραίτητο για τη σωστή λειτουργία της AND-αναζήτησης και της λίστας δημοφιλών ετικετών σε επίπεδο ολόκληρης της πλατφόρμας.
 - Δημόσιο περιεχόμενο. Η εκφώνηση αναφέρει ρητά ότι όλα τα άλμπουμ και φωτογραφίες είναι δημόσια, οπότε δεν υλοποίησα κανέναν μηχανισμό ορατότητας ή ιδιωτικότητας.
 - Βαθμολογία συνεισφοράς. Ορίζεται ως φωτογραφίες που ανέβηκαν συν σχόλια σε φωτογραφίες άλλων χρηστών. Το SQL ερώτημα φιλτράρει ρητά τα σχόλια που άφησε ο χρήστης σε φωτογραφίες των δικών του άλμπουμ.
 - Αναζήτηση σχολίων με ακριβή αντιστοίχιση. Η εκφώνηση αναφέρει «ταιριάζουν ακριβώς με το κείμενο του ερωτήματος». Υλοποίησα αυτό ως `WHERE content = query` — ακριβής σύγκριση, όχι LIKE ή partial match.
 - Αποθήκευση κωδικών με κρυπτογράφηση. Οι κωδικοί αποθηκεύονται ως bcrypt hashes μέσω της `generate_password_hash()` της βιβλιοθήκης werkzeug. Κανένας κωδικός δεν αποθηκεύεται σε απλό κείμενο.
 - Δυαδικά δεδομένα εικόνας σε BYTEA. Η εκφώνηση απαιτεί αποθήκευση της πραγματικής δυαδικής αναπαράστασης. Το Flask διαβάζει τα bytes του ανεβασμένου αρχείου και τα περνά στο `Binary()` wrapper του pycorg2. Ο τύπος MIME εντοπίζεται αυτόματα από τα magic bytes κατά την εμφάνιση, υποστηρίζοντας PNG, JPEG, GIF και WebP.
-

5. Οδηγίες Εγκατάστασης και Setup

Το project απαιτεί Python 3, PostgreSQL και pip. Πριν οποιαδήποτε εντολή, χρειάζεται να ανοίξετε το αρχείο `db.py` και να ορίσετε το πεδίο `user` στο `DB_CONFIG` με το τοπικό PostgreSQL username σας.

Επιλογή A — Αυτόματη εγκατάσταση (συνιστάται)

Από τον κατάλογο `k29photo/` εκτελέστε:

```
bash setup.sh
```

Το script εκτελεί αυτόματα τα εξής βήματα: ελέγχει την ύπαρξη των εξαρτήσεων (`psql`, `python3`, `pip3`), εγκαθιστά τις Python βιβλιοθήκες, διαγράφει τυχόν υπάρχουσα βάση `k29photo` και την αναδημιουργεί από την αρχή, φορτώνει το `schema.sql` και στη συνέχεια το `data.sql`, εκτυπώνει σύνοψη με τον αριθμό εγγραφών κάθε πίνακα, και τέλος εκκινεί το Flask portal.

Επιλογή B — Χειροκίνητη εγκατάσταση

- `pip install flask pycorg2-binary werkzeug`
- `createdb k29photo`
- `psql -d k29photo -f schema.sql`
- `psql -d k29photo -f data.sql`

python3 [app.py](#)

Στη συνέχεια ανοίξτε το <http://localhost:5000> σε browser.

Διαπιστευτήρια δεδομένων δείγματος

Η βάση προφορτώνεται με 12 χρήστες, 16 άλμπουμ, 30 φωτογραφίες, 20 ετικέτες, 32 φιλίες, 40 σχόλια και 54 likes. Όλοι οι χρήστες του δείγματος έχουν κωδικό password123. Παράδειγμα σύνδεσης: nikos.p@gmail.com / password123.

6. Υλοποίηση Περιπτώσεων Χρήσης

#	Περίπτωση Χρήσης	Route	Αρχείο
1α	Εγγραφή χρήστη + σφάλμα διπλότυπου email	POST /register	auth.py
1α	Σύνδεση / Αποσύνδεση	POST /login, GET /logout	auth.py
1β	Προσθήκη φίλου, αναζήτηση χρηστών, λίστα φίλων	GET/POST /friends	friends.py
1γ	Κατάταξη 10 κορυφαίων συνεισφερόντων	GET /activity	main.py
2α	Περιήγηση άλμπουμ & φωτογραφιών (επισκέπτες)	GET /browse, /albums/<id>	main.py, albums.py
2β	Δημιουργία & διαγραφή άλμπουμ	POST /albums/create, /delete	albums.py
2β	Ανέβασμα & διαγραφή φωτογραφίας	POST /photos/upload, /delete	photos.py
3α	Κλικάρισιμες ετικέτες → φωτογραφίες με αυτή	GET /tags/<name>	tags.py
3β	Toggle Όλες Φωτογραφίες / Δικές μου	GET /tags/<name>?view=all mine	tags.py

3γ	Δημοφιλέστερες ετικέτες, κλικάρισιμες	GET /search (Discover page)	photos.py
3δ	AND αναζήτηση ετικετών, επισκέπτες & χρήστες	GET /search?q=tag1+tag2	photos.py
4α	Δημοσίευση σχολίου, επισκέπτης & εγγεγραμμένος	POST /photos/<id>/comment	comments.py
4α	Αποκλεισμός αυτο-σχολίου μέσω trigger	BEFORE INSERT trigger	schema.sql
4β	Like / unlike φωτογραφίας, εμφάνιση likers	POST /photos/<id>/like	photos.py
4γ	Αναζήτηση σχολίων ακριβούς αντιστοίχισης	GET /search/comments	comments.py
5α	Συστάσεις φίλων μέσω friends-of-friends	GET /recommendations	recommendations.py
5β	Λειτουργία "You-may-also-like"	GET /recommendations	recommendations.py

7. Παραλείψεις και Παρατηρήσεις

- Δεν υλοποιήθηκε σελιδοποίηση (pagination). Τα photo grids φορτώνουν όλα τα αποτελέσματα. Για σύστημα παραγωγής θα προσθέταμε LIMIT/OFFSET pagination για αποφυγή προβλημάτων απόδοσης με μεγάλα σύνολα δεδομένων.
- Δεν υλοποιήθηκε επαλήθευση τύπου εικόνας. Το route ανεβάσματος δέχεται οποιονδήποτε τύπο αρχείου. Ο τύπος MIME εντοπίζεται κατά την εμφάνιση από τα magic bytes, αλλά δεν υπάρχει server-side validation κατά το upload.
- Δεν υλοποιήθηκαν σελίδες προφίλ χρηστών. Οι χρήστες μπορούν να διαχειρίζονται τα δικά τους άλμπουμ, αλλά δεν υπάρχει δημόσια σελίδα προφίλ. Αυτό δεν ζητήθηκε στην εκφώνηση.
- Αντίφαση εκφώνησης στα σχόλια επισκεπτών. Η περιγραφή δεδομένων ορίζει τον ιδιοκτήτη σχολίου ως νόμιμο χρήστη, ενώ η περίπτωση χρήσης 4α επιτρέπει ρητά σχόλια επισκεπτών. Ακολούθησα τον ορισμό της περίπτωσης χρήσης ως πιο ειδικό, και υλοποίησα nullable user_id με εναλλακτικό πεδίο guest_name.
- Αναζήτηση σχολίων με ακριβή αντιστοίχιση. Σύμφωνα με την εκφώνηση («ταιριάζουν ακριβώς»), η αναζήτηση χρησιμοποιεί WHERE content = query. Αυτό σημαίνει ότι ο χρήστης πρέπει να πληκτρολογήσει το ακριβές κείμενο του σχολίου για να λάβει αποτελέσματα —

συμπεριφορά που ακολουθεί τις οδηγίες αλλά είναι λιγότερο φιλική προς τον χρήστη από μια partial search.